

Base de Datos Distribuidas

SEMANA 13

Heider Sanchez
hsanchez@utec.edu.pe

4.



Base de Datos Distribuidas

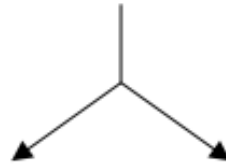
Fragmentación Vertical

Fragmentación Vertical

- Ejemplo:

E

id	name	location	salary
1	Tom	A	15
2	Ann	B	23
3	Ben	A	21



E₁

id	name	location
1	Tom	A
2	Ann	B
3	Ben	A

E₂

id	salary
1	15
2	23
3	21

Fragmentación Vertical

- Es parecido a la normalización de relaciones (formas normales).

$$\begin{array}{c} R [A] \Rightarrow R_1 [A_1] \\ \vdots \\ R_n [A_n] \end{array}$$

Conjunto de atributos

$$A = \{ a_1, a_2, \dots, a_n \}$$

$$A_i \subseteq A$$

Fragmentación Vertical

- **Propiedades:**

- Sea:

$$R[A] \Rightarrow R_i[A_i]$$

- (1) Completitud:

- Se cumple que:

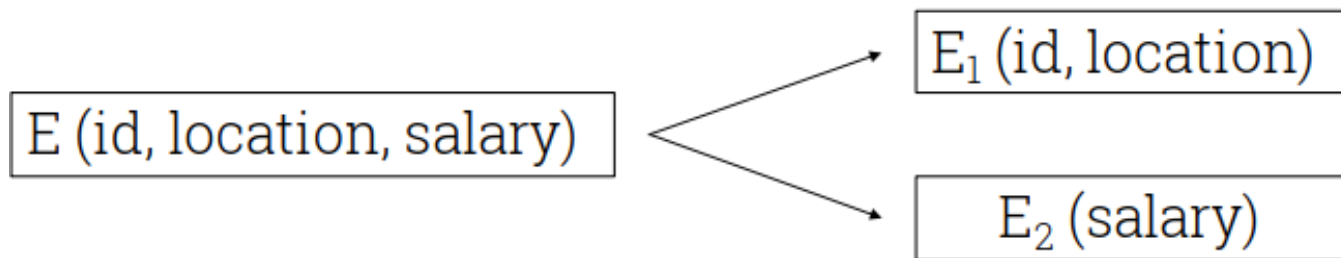
$$\bigcup A_i = A \quad \forall i$$

Fragmentación Vertical

- **Propiedades:**

- (2) Desarticulación:

$$A_i \cap A_j = \emptyset \quad \forall i, j: i \neq j$$

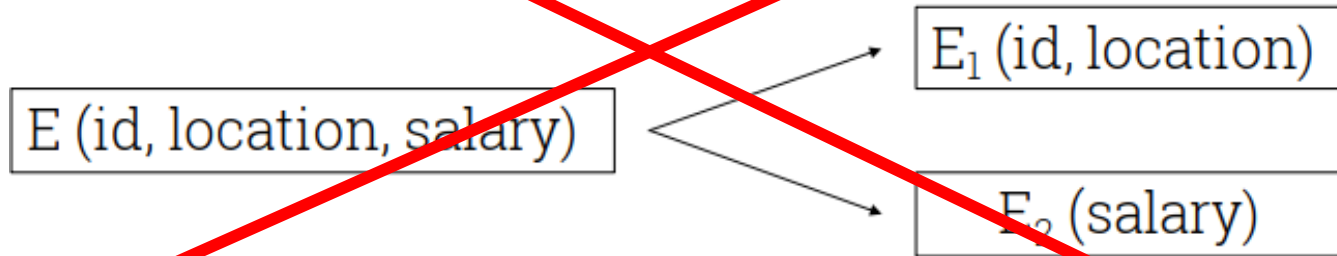


Fragmentación Vertical

- **Propiedades:**

- (2) Desarticulación:

$$A_i \cap A_j = \emptyset \quad \forall i, j: i \neq j$$



No es una propiedad deseable
¿Por qué?

Fragmentación Vertical

- **Propiedades:**

- (2) Join sin perdida:

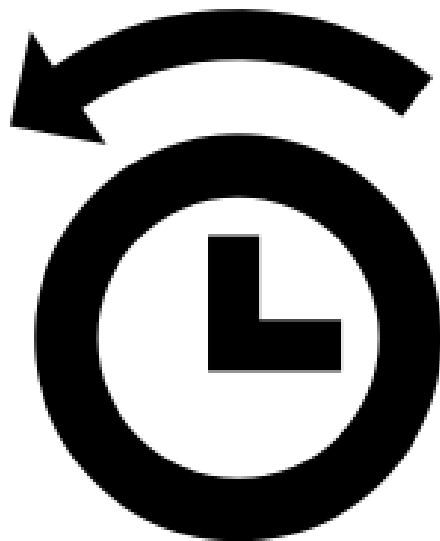
$$\bowtie R_i = R \quad \forall i$$

- Debe ser posible la reconstrucción total de R sin tuplas falsas y sin perdida de tuplas.
- Formas de lograr el Join sin perdida:
 - a. Dependencias funcionales
 - b. Matriz de afinidad de atributos



Fragmentación por dependencias funcionales





Dependencias funcionales

Dada una relación R

y dos conjuntos de atributos $X \in R, Y \in R$

X determina funcionalmente a Y

si y sólo si

dos tuplas que tiene el mismo valor de X

deben por fuerza tener el mismo valor de Y

- X y Y pueden ser atributos **compuestos**

Dependencias funcionales: Ejemplo 1

Sea la instancia (representativa):

IMPARTIR

Profesor	Curso	Texto
Juan	Estructuras de datos	Bartram
Juan	Administración	Martín
Pedro	Compiladores	Hoffman
Celis	Estructura de datos	Horowitz
Pedro	Administración	Martín

{Texto} → {Curso}

{Profesor} → {Curso}

Descomposición

Definición

Sea la relación **R** y su conjunto de dependencias funcionales **F**

Una descomposición **R1** y **R2** es sin pérdida si:

- Conservan los atributos de R
 - $R = R1 \cup R2$
- Conservan las dependencias funcionales
 - $F^+ = \{F1 \cup F2\}^+$
- No genera tuplas falsas, para ello:
 - $\{R1 \cap R2\} \rightarrow R1$ ó
 - $\{R1 \cap R2\} \rightarrow R2$

* La intersección genera una dependencia funcional válida en F

Descomposición

Ejemplo 1

Sea $R(A,B,C,D,E)$ y $F=\{AB \rightarrow C, C \rightarrow D, C \rightarrow E\}$

La descomposición $R1(A,B,C,D)$ y $R2(C,E)$

genera $F1=\{AB \rightarrow C, C \rightarrow D\}$ y $F2=\{C \rightarrow E\}$

- ¿Conservan los atributos de R?
 - Si
- ¿Conserva las dependencias funcionales?
 - Si
- ¿No genera tuplas falsas?:
 - No
 - $\{R1 \cap R2\} \rightarrow R2$
 - $C \rightarrow CE \subset F^+$

Descomposición

Ejemplo 2

Sea $R(\underline{A}, \underline{B}, C, D, E, F)$ y $F = \{AB \rightarrow D, AC \rightarrow E, ABD \rightarrow F\}$

La descomposición $R_1(\underline{A}, \underline{B}, D, F)$ y $R_2(\underline{A}, \underline{C}, E)$
genera $F_1 = \{AB \rightarrow D, ABD \rightarrow F\}$ y $F_2 = \{AC \rightarrow E\}$

- Si conservan los atributos de R
- Si conserva las dependencias funcionales
- **SI genera tuplas falsas, ya que:**
 - $\{R_1 \cap R_2\} \rightarrow R_1: A \rightarrow BDF \not\subseteq F^+$
 - $\{R_1 \cap R_2\} \rightarrow R_2: A \rightarrow CE \not\subseteq F^+$

Descomposición en 3FN

- Definición
 - Está en 2FN
 - Para toda DF $X \rightarrow Y$ en R:
 - X es superclave ó Y es atributo primo
- Descomposición
 - $R(A, X, Y, B)$ donde $X \rightarrow Y$ incumple 3FN
 - Crear otra relación con X^+ , donde X es clave
 - Eliminar Y de R

Descomposición en 3FN

Ejemplo

- Sea $R(\underline{A}, X, Y, B)$ y $F = \{A \rightarrow XB, X \rightarrow Y\}$.
 - No es 3FN, X no es superclave
- Tomando las F como base:
 - $R_1(\underline{A}, X, B)$
 - $R_2(\underline{X}, Y)$
 - $(R_1 \cap R_2 \rightarrow R_2) \subset F^+$

Normalización en 3FN

- Descomposición
 - Calcular F mínimo
 - Convertir cada dependencia en una relación
$$(X \rightarrow Y \Rightarrow R_i(XY))$$
 - Si no está la llave en una relación, agregarla
 - Eliminar relaciones redundantes

Normalización en 3FN

Ejemplo

- Sea $R(A, B, C, D, E)$ y $F = \{A \rightarrow B, A \rightarrow C, C \rightarrow D, B \rightarrow E\}$.
 - F es mínimo
- $\{A\}^+ = \{A, B, C, D, E\} \Rightarrow A$ es clave única
- Tomando las cuatro DFs:

Normalización en 3FN

Ejemplo

- Sea $R(A, B, C, D, E)$ y $F = \{A \rightarrow B, A \rightarrow C, C \rightarrow D, B \rightarrow E\}$.
 - F es mínimo
- $\{A\}^+ = \{A, B, C, D, E\} \Rightarrow A$ es clave única
- Tomando las cuatro DFs:
 - ~~$R_1(A, B)$~~ ,
 - ~~$R_2(A, C)$~~ ,
 - $R_3(A, C, D)$
 - $R_4(A, B, E)$

Normalización en FNBC

- Definición
 - Está en 3FN (... válido desde 1FN)
 - Para toda DF $X \rightarrow Y$ en R:
 - X es superclave
- Descomposición
 - $R(A,X,Y,B)$ donde $X \rightarrow Y$ incumple FNBC
 - Crear dos relaciones
 - $R1 = R - Y$
 - $R2(X,Y)$

Esta estrategia de normalización no asegura preservar dependencias, pero si asegura la recuperación de la información por join.

Normalización en FNBC

Ejemplo

Sea $R(A, B, C, D, E)$ y $F = \{A \rightarrow BC, C \rightarrow D, B \rightarrow E\}$.

1- R no está en FNBC, $C \rightarrow D$ incumple FNBC

Partimos: $R(A, B, C, E)$ y $R_1(C, D)$.

2- R no está en FNBC, $B \rightarrow E$ incumple FNBC

Partimos: $R(A, B, C)$ y $R_2(B, E)$

Resultado: $R_1(C, D)$, $R(A, B, C)$ y $R_2(B, E)$



Fragmentación por afinidad de atributos



Fragmentación Vertical

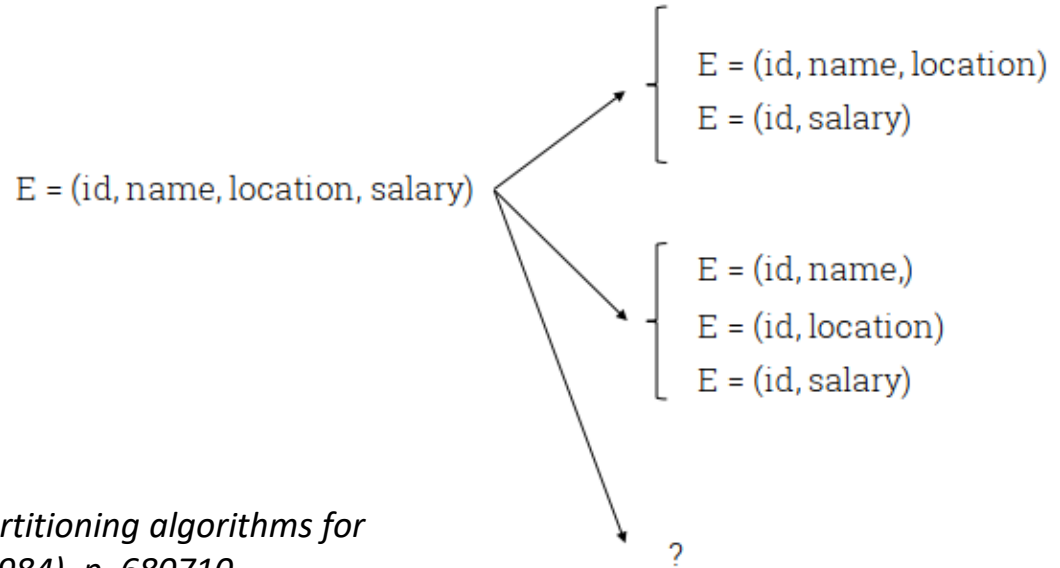
- **Matriz de afinidad de atributos:**

1. Repetir la clave en todos los fragmentos.

$$\text{key} \subseteq A_i \quad \forall i$$

2. Agrupar atributos por afinidad

- ¿Cómo decidir
que atributos agrupar?



Fragmentación Vertical

- **Vertical Clustering and Bond Energy Algorithm (BEA)**
 - El objetivo es agrupar atributos que tienen alta afinidad entre sí, es decir, aquellos que suelen ser consultados de manera conjunta.

<https://infolab.usc.edu/csci585/Spring2008/Lectures/DDBMS-Design.pdf>

P. Arabie and L. J. Hubert, "The bond energy algorithm revisited," in IEEE Transactions on Systems, Man, and Cybernetics, vol. 20, no. 1, pp. 268-274, Jan.-Feb. 1990.

Fragmentación Vertical

- **Matriz de afinidad de atributos:**

- Dado un conjunto de consultas $\{q_1, q_2, \dots\}$ que se ejecutan sobre la relación $R[A_1, A_2, \dots]$

- $$\text{use}(q_k, A_j) = \begin{cases} 1 & \text{si } A_j \text{ es referenciado por } q_k \\ 0 & \text{en otro caso} \end{cases}$$

	A_1	A_2	A_3	A_4
q_1	1	0	1	0
q_2	0	1	1	0
q_3	0	1	0	1
q_4	0	0	1	1

Fragmentación Vertical

- **Matriz de afinidad de atributos:**

- Paso 1: Se crea una matriz que representa la frecuencia con la que los atributos son accedidos juntos en consultas (por las distintas aplicaciones):

$$Aff(A_i, A_j) = \sum_{k | use(q_k, A_i)=1 \wedge use(q_k, A_j)=1} \sum_{\forall l} ref_l(q_k) * acc_l(q_k)$$

- En donde:
 - $ref_l(q_k)$ indica el número de accesos a los atributos (A_i, A_j) por la consulta q_k en el sitio l .
 - $acc_l(q_k)$ indica la frecuencia de ejecución de la consulta q_k en el sitio l .

Fragmentación Vertical

- **Matriz de afinidad de atributos:**

- Ejemplo:

- Sea $ref_l(q_k) = 1$ para todo q_k en el sitio l . La frecuencia de ejecución es:

$$\text{Site 1} \left\{ \begin{array}{l} acc_1(q_1)=15 \\ acc_1(q_2)=5 \\ acc_1(q_3)=25 \\ acc_1(q_4)=3 \end{array} \right. \text{Site 2} \left\{ \begin{array}{l} acc_2(q_1)=20 \\ acc_2(q_2)=0 \\ acc_2(q_3)=25 \\ acc_2(q_4)=0 \end{array} \right. \left. \begin{array}{l} acc_3(q_1)=10 \\ acc_3(q_2)=0 \\ acc_3(q_3)=25 \\ acc_3(q_4)=0 \end{array} \right\} \text{Site 3}$$

- $aff(A1, A3) = acc_1(q_1) + acc_2(q_1) + acc_3(q_1) = 45$, dado que q_1 que es la única consulta que accede a ambos atributos.
 $aff(A2, A4) = acc_1(q_3) + acc_2(q_3) + acc_3(q_3) = 75$

Fragmentación Vertical

- **Matriz de afinidad de atributos:**

- Entonces la matriz quedaría de la siguiente forma:

$$\begin{array}{c} q_1 \\ q_2 \\ q_3 \\ q_4 \end{array} \begin{array}{c} A_1 \quad A_2 \quad A_3 \quad A_4 \\ \left(\begin{array}{cccc} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{array} \right) \end{array}$$

matriz de uso

$$\begin{array}{c} A_1 \\ A_2 \\ A_3 \\ A_4 \end{array} \begin{array}{c} A_1 \quad A_2 \quad A_3 \quad A_4 \\ \left(\begin{array}{cccc} 45 & 0 & 45 & 0 \\ 0 & 80 & 5 & 75 \\ 45 & 5 & 53 & 3 \\ 0 & 75 & 3 & 78 \end{array} \right) \end{array}$$

matriz de afinidad

- **Bond Energy Algorithm (BEA)**

```

Algorithm 2.3: BEA


---


Input:  $AA$ : attribute affinity matrix
Output:  $CA$ : clustered affinity matrix
begin
    {initialize; remember that  $AA$  is an  $n \times n$  matrix}
     $CA(\bullet, 1) \leftarrow AA(\bullet, 1)$ 
     $CA(\bullet, 2) \leftarrow AA(\bullet, 2)$ 
     $index \leftarrow 3$ 
    while  $index \leq n$  do                                {choose the “best” location for attribute  $AA_{index}$ }
        for  $i$  from 1 to  $index - 1$  by 1 do calculate  $cont(A_{i-1}, A_{index}, A_i)$ 
        calculate  $cont(A_{index-1}, A_{index}, A_{index+1})$                 {boundary condition}
         $loc \leftarrow$  placement given by maximum  $cont$  value
        for  $j$  from  $index$  to  $loc$  by  $-1$  do
             $CA(\bullet, j) \leftarrow CA(\bullet, j - 1)$                 {shuffle the two matrices}
        end for
         $CA(\bullet, loc) \leftarrow AA(\bullet, index)$ 
         $index \leftarrow index + 1$ 
    end while
    order the rows according to the relative ordering of columns
end

```

Input: AA : attribute affinity matrix

Output: CA : clustered affinity matrix

begin

{initialize; remember that AA is an $n \times n$ matrix}

$$CA(\bullet, 1) \leftarrow AA(\bullet, 1)$$
$$CA(\bullet, 2) \leftarrow AA(\bullet, 2)$$
$$index \leftarrow 3$$

while $index \leq n$ **do** {choose the “best” location for attribute AA_{index} }

for i from 1 to $index - 1$ by 1 **do** calculate $cont(A_{i-1}, A_{index}, A_i)$

calculate $cont(A_{index-1}, A_{index}, A_{index+1})$ {boundary condition}

$$loc \leftarrow \text{placement given by maximum } cont \text{ value}$$
for j *from* $index$ *to* loc *by* -1 **do**
$$| \quad \tilde{CA}(\bullet, j) \leftarrow CA(\bullet, j-1) \quad \{\text{shuffle the two matrices}\}$$
end for
$$CA(\bullet, loc) \leftarrow AA(\bullet, index)$$
$$index \leftarrow index + 1$$
end while

order the rows according to the relative ordering of columns

end

Fragmentación Vertical

- **Matriz de afinidad de atributos:**

	A ₁	A ₂	A ₃	A ₄
A ₁	45	0	45	0
A ₂	0	80	5	75
A ₃	45	5	53	3
A ₄	0	75	3	78

	A1	A3	A2	A4
A1	45	45	0	
A2	0	5	80	
A3	45	53	5	
A4	0	3	75	

- Determinar el orden de A3

$$cont(A_i, A_k, A_j) = 2 \cdot \{bond(A_i, A_k) + bond(A_k, A_j) - bond(A_i, A_j)\}$$

$$bond(A_x, A_y) = \sum_{z=1}^n aff(A_z, A_x) \cdot aff(A_z, A_y)$$

Fijamos A1, A2, y comenzamos el analisis desde A3

$$Cont(A0, \mathbf{A3}, \mathbf{A1}) = 8820 \quad Cont(\mathbf{A1}, \mathbf{A3}, \mathbf{A2}) = 10150 \quad Cont(\mathbf{A2}, \mathbf{A3}, \mathbf{A5}) = 1780$$

Dado que $Cont(A1, A3, A2)$ es el más grande, entonces $[A1, A3, A2]$ es el mejor orden.

Condición de contorno:

$$aff(A0, A_j) = aff(A_i, A0) = aff(A_{n+1}, A_j) = aff(A_i, A_{n+1}) = 0$$

Fragmentación Vertical

- **Matriz de afinidad de atributos:**

- Determinar el orden de A3

	A ₁	A ₂	A ₃	A ₄
A ₁	45	0	45	0
A ₂	0	80	5	75
A ₃	45	5	53	3
A ₄	0	75	3	78

	A1	A3	A2	A4
A1	45	45	0	
A2	0	5	80	
A3	45	53	5	
A4	0	3	75	

	A ₁	A ₃	A ₂	A ₄
A ₁	45	45	0	
A ₂	0	5	80	
A ₃	45	53	5	
A ₄	0	3	75	

[A1 , A3, A2] es el mejor orden.

Cont(A0,A4,A1) =

Cont(A1,A4,A3) =

Cont(A3,A4,A2) =

Cont(A2,A4,A5) =

Fragmentación Vertical

- **Matriz de afinidad de atributos:**

- Determinar el orden de A4

	A1	A3	A2	A4
A1	45	45	0	0
A3	45	53	5	3
A2	0	5	80	75
A4	0	3	75	78

$$\begin{matrix} & A_1 & A_3 & A_2 & A_4 \\ A_1 & 45 & 45 & 0 & 0 \\ A_2 & 0 & 5 & 80 & 75 \\ A_3 & 45 & 53 & 5 & 3 \\ A_4 & 0 & 3 & 75 & 78 \end{matrix}$$

[A2, A4, A5] es el mejor orden.

Fragmentación Vertical

- **Matriz de afinidad de atributos:**

- Determinar el orden de A4

	A1	A3	A2	A4
A1	45	45	0	0
A3	45	53	5	3
A2	0	5	80	75
A4	0	3	75	78

$$\begin{matrix} & A_1 & A_3 & A_2 & A_4 \\ A_1 & 45 & 45 & 0 & 0 \\ A_2 & 0 & 5 & 80 & 75 \\ A_3 & 45 & 53 & 5 & 3 \\ A_4 & 0 & 3 & 75 & 78 \end{matrix}$$

[A2, A4, A5] es el mejor orden.

Fragmentación Vertical

- **Matriz de afinidad de atributos:**

- Las filas son organizados en el mismo orden que las columnas

	A_1	A_2	A_3	A_4
A_1	45	0	45	0
A_2	0	80	5	75
A_3	45	5	53	3
A_4	0	75	3	78



	A_1	A_3	A_2	A_4
A_1	45	45	0	0
A_3	45	53	5	3
A_2	0	5	80	75
A_4	0	3	75	78

Matriz de afinidad agrupada

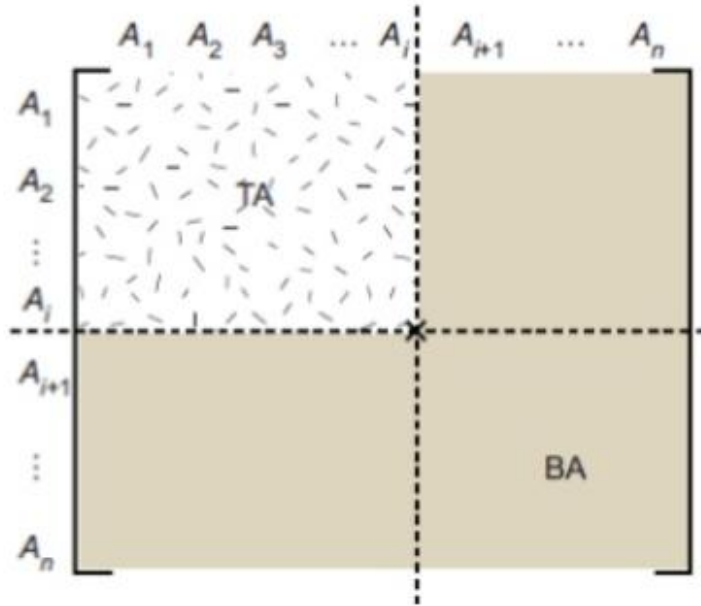
Fragmentación Vertical

- **Matriz de afinidad de atributos:**

- Paso 3: encuentre los conjuntos de atributos a los que se accede, en su mayor parte, por distintos conjuntos de aplicaciones (ejecución de consultas).
 - Se busca puntos de división a lo largo de la diagonal de manera que:
 - Se maximicen los accesos totales a un solo fragmento, mientras que
 - Se minimicen los accesos totales a más de un fragmento.

Fragmentación Vertical

- Matriz de afinidad de atributos:



Se traduce en maximizar z

$$z = CTA * CBA - COQ^2$$

CTA = Suma total de afinidad en el cuadrante TA

CBA = Suma total de afinidad en el cuadrante BA

COQ = Suma total de afinidad en los cuadrantes restantes

Fragmentación Vertical

- Matriz de afinidad de atributos:

	A_1	A_3	A_2	A_4
A_1	45	45	0	0
A_3	45	53	5	3
A_2	0	5	80	75
A_4	0	3	75	78

$$R_1 = [k, A_1, A_3]$$

$$R_2 = [k, A_2, A_4]$$